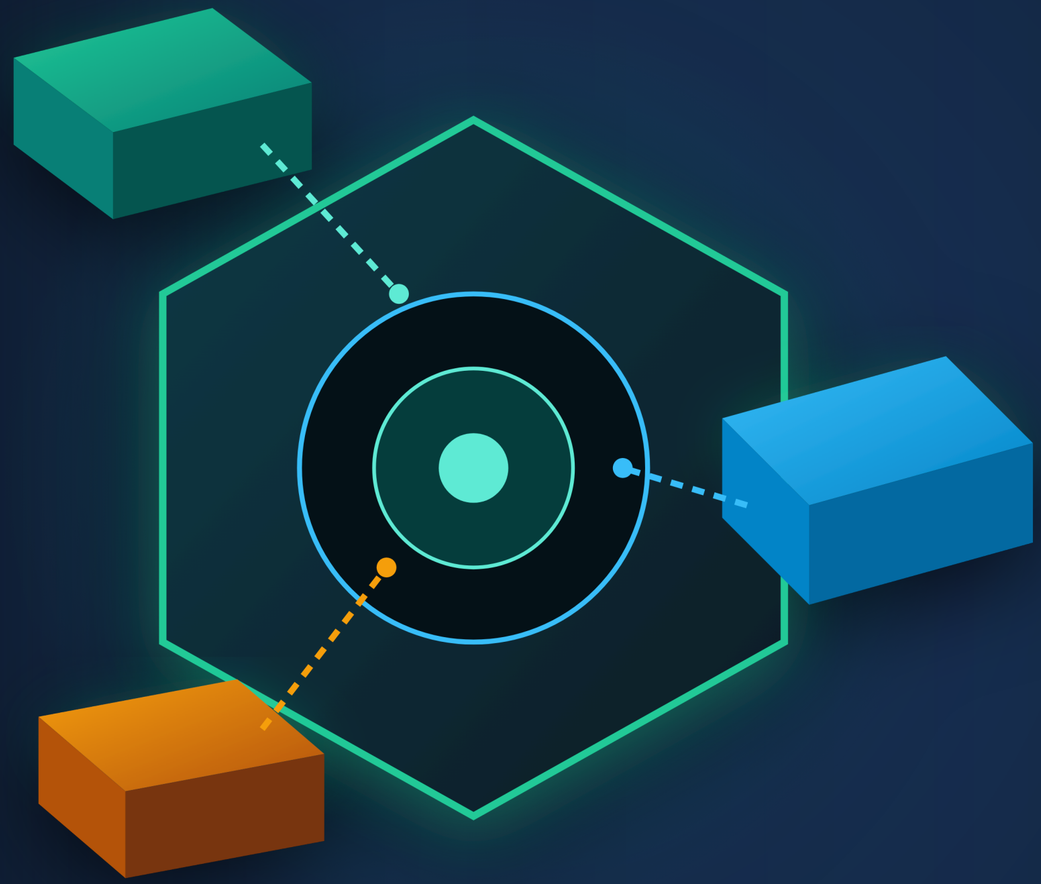
SOFTWARE ENGINEERING Lab 16

Dependency Injection & Inversion of Control

Memisahkan Konfigurasi dari Penggunaan:
Constructor Injection vs Service Locator



Lukman / lukman-ss

Coupling Keras Merusak Testability & Evolusi

Instansiasi dependensi langsung di dalam logika bisnis mengunci sistem ke infrastruktur spesifik.

: ()
: ()
: ()

Ketika komponen bisnis menginstansiasi dependensinya sendiri, kode menjadi terkunci pada infrastruktur konkret. Pengujian terisolasi menjadi mustahil dan pergantian vendor memicu refactor besar.

Hard-coded dependencies fuse business logic directly to infrastructure.

Mengapa Global State & Manual Swap Gagal

Pendekatan naif seperti singleton global atau service locator justru menambah kerapuhan baru.

: ()
: ()
: ()

Mencoba menyelesaikan coupling dengan Service Locator atau Singleton global hanya memindahkan masalah: dependensi menjadi tersembunyi dan runtime error menggantikan compile-time error.

Service Locator trades explicit compile-time coupling for hidden runtime lookup failures.

Pemisahan Konfigurasi dari Penggunaan

Martin Fowler: Memisahkan perakitan objek dari eksekusi bisnis lebih mendasar dibanding pilihan DI.

: ()
: ()
: ()

Inti DI adalah memisahkan dua siklus hidup: pembuatan grafik objek di satu composition root, dan eksekusi logika domain yang hanya berinteraksi melalui kontrak interface.

Separate construction from consumption: objects receive dependencies, never build them.

Alur Dependensi: Composition Root ke Gateway

Kontrak interface membalikkan kendali implementasi dari consumer ke composition root.

STEP 01

cmd/demo/main.go

Composition root: inisialisasi RealGateway & Processor

→

STEP 02

PaymentGateway Interface

Kontrak abstrak: Charge(Money) error

→

STEP 03

Processor.ProcessPayment

Validasi domain amount > 0 & bungkus Money

→

STEP 04

MockGateway / RealGateway

Eksekusi jaringan nyata atau mock test terisolasi

Alur kerja DI menghubungkan composition root ke gateway melalui kontrak interface. Processor hanya tahu interface PaymentGateway, sehingga implementasi konkret dapat diganti sesuka hati.

High-level modules depend on abstractions, decoupling business rules from network adapters.

Constructor Injection: Valid at Birth

Signature konstruktor secara eksplisit menuntut dependensi kontrak yang wajib diisi.

```
type Processor struct {
    gateway PaymentGateway // unexported field
}

func NewProcessor(g PaymentGateway) *Processor {
    return &Processor{gateway: g}
}

func (p *Processor) ProcessPayment(amount int) error {
    if amount <= 0 { return errors.New("invalid amount") }
    return p.gateway.Charge(Money{Amount: amount, Currency: "USD"})
}
```

Constructor Injection memastikan objek selalu berada dalam state valid sejak dibuat. Tidak ada hidden dependency, tidak ada container import, dan compile-time type safe.

Explicit constructor parameters guarantee valid object state at instantiation.

internal/di/processor.go:6–21

Failure Path: Validasi Dini Mencegah I/O Sia-sia

Pre-check input memotong eksekusi sebelum memicu panggilan jaringan infrastruktur.

```
Client Input: ProcessPayment(-10)
  -> Guard Clause: if amount <= 0 (VALIDATION FAILURE)
  -> Return: errors.New("invalid amount")
  -> Short-Circuit: Gateway.Charge() TIDAK PERNAH DIPANGGIL

Gateway Down: MockGateway.ShouldFail = true
  -> Input Valid: ProcessPayment(50) -> Gateway.Charge(50 USD)
  -> Infrastructure Error: "gateway unavailable" dipropagasi utuh
```

DI tidak menghilangkan validasi. Guard clause domain memastikan panggilan jaringan tidak pernah dieksekusi jika input tidak valid, sementara error infrastruktur dipropagasikan secara transparan.

Domain pre-conditions protect infrastructure boundaries from invalid execution.

internal/di/processor.go:15-21 • tests/processor_test.go:47-59

Tabel Komparasi: Tiga Pola Pengelolaan Dependensi

Perbandingan mendasar antara Hard-coded, Service Locator, dan Constructor Injection.

Dimensi	Hard-Coded (new)	Service Locator	Constructor Injection
Visibilitas Dependensi	Terkunci internal	Opaque / Tersembunyi	Eksplisit di constructor
Validasi Compile-Time	Ya (tapi kaku)	Tidak (runtime lookup)	Ya (interface-checked)
Isolasi Unit Test	Sangat Sulit	Perlu Mock Container	Mudah (Mock Interface)
Kopling ke Framework	Nol	Tinggi (Container API)	Nol (POPO / Standard Go)

Tabel ini memetakan trade-off ketiga pola. Constructor Injection memberikan keseimbangan terbaik: dependensi terlihat jelas di signature tanpa mengikat kode ke framework container.

Explicit interfaces beat both rigid hardcoding and opaque container lookups.

Decision Sequence: Kapan Menggunakan Pola Tertentu

Tiga keputusan arsitektural untuk menentukan strategi instansiasi objek.

: ()
: ()
: ()

Jangan perlakukan semua objek sama. Service dengan I/O eksternal membutuhkan Constructor Injection, sedangkan value object murni harus diinstansiasi langsung tanpa perantara.

Inject infrastructure boundaries; instantiate domain value objects directly.

Bukti Eksekusi: Demo Wiring & Test Suite

Verifikasi runtime membuktikan kedua pola bekerja fungsional, namun dengan ergonomi berbeda.

```
$ go run ./cmd/demo
--- Running Constructor Injection ---
RealGateway charging 100 USD
--- Running Service Locator ---
RealGateway charging 200 USD

$ go test -v -count=1 ./...
PASS: TestProcessor_Success (0.00s) | TestProcessor_GatewayError (0.00s)
PASS: TestProcessor_InvalidAmount (0.00s) | TestBadProcessor_Success (0.00s)
ok   lab16/tests   0.387s (all 6 tests PASS, race-clean)
```

Output demo dan test suite membuktikan kedua pola berfungsi, namun Constructor Injection mencapai hasil yang sama dengan nol dependensi terhadap container.

Both patterns execute identically, but constructor injection eliminates runtime container friction.

Analisis Batasan Bukti: Fakta vs Ekstrapolasi

Memisahkan temuan empiris laboratorium dari asumsi industri yang belum teruji di lab.

YANG DIBUKTIKAN LAB

Mock injection mengisolasi test tanpa network call (0.00s).
Constructor injection menjamin valid-at-birth compile-time.
Guard clause mencegah pemanggilan gateway saat input invalid.
Value object (Money) dapat dipakai langsung tanpa DI.

YANG TIDAK DIBUKTIKAN

Lifecycle container (singleton vs scoped vs transient).
Performa overhead auto-wiring framework (wire / dig).
Klaim penurunan defect rate sistem production secara empiris.
Batas 12 parameter constructor (hanya heuristik lab).

Lab membuktikan isolasi test dan pemisahan perakitan, tetapi tidak membuktikan siklus hidup scoped/transient container maupun metrik penurunan bug di production.

Scope claims strictly to verified test doubles; do not extrapolate lifecycle benchmarks.

Trade-off Arsitektur: Eksplisit vs Fleksibilitas

Constructor injection mengorbankan keringkasan kode awal demi transparansi dan keandalan.

KEUNTUNGAN (CONSTRUCTOR DI)

Transparansi total: semua kebutuhan terlihat di signature.
Immutabilitas: dependensi disetel saat inisialisasi dan unexported.
Compile-time safety: compiler Go menolak jika dependensi kurang.

KONSEKUENSI / BIAYA

Boilerplate konstruktor bertambah di composition root (main.go).
Perubahan dependensi memerlukan modifikasi signature konstruktor.
Tracing alur kode memerlukan navigasi manual tanpa magic wiring.

Constructor Injection tidak gratis: ia menambah boilerplate di entry point. Namun biaya ini sebanding dengan jaminan type-safety dan kemudahan pembacaan dependensi.

Pay upfront boilerplate in main.go to buy absolute clarity and testability across domain services.

Pipeline Validasi & Keamanan Konkurensi

Struct immutable dan stateless service memastikan eksekusi aman di multi-goroutine.

```
Goroutine Client Request -> NewProcessor(mock)
  -> ProcessPayment(amount) [Stateless Validation]
  -> if amount <= 0 -> Instant Error (No I/O)
  -> Value Object: Money{Amount, USD} [Immutable, Stack-allocated]
  -> Gateway.Charge(Money) [Thread-Safe Interface Call]
Race Detector Verification: go test -race ./... -> 0 warnings
```

Dengan menginjeksi interface stateless dan memakai value object immutable, Processor bebas dari race condition saat diakses oleh banyak goroutine secara bersamaan.

Immutable dependencies and stack-allocated value objects yield natural thread safety.

Sebelum vs Sesudah Constructor Injection

Transformasi dari tight coupling ke decoupled architecture berbasis interface.

BEFORE: NAIVE / UNPROTECTED

SEBELUM: SERVICE LOCATOR / TIGHT

- Processor memanggil new RealGateway() atau container.Get().
- Test unit wajib menjalankan server mock HTTP atau mock container.
- Signature NewBadProcessor(Container) menyembunyikan kebutuhan gateway.
- Missing dependency meledak di runtime saat method dipanggil.

AFTER: OPTIMIZED / RESILIENT

SESUDAH: CONSTRUCTOR INJECTION

✓ Processor hanya bergantung pada interface PaymentGateway.

✓ Test unit menyuntikkan MockGateway in-memory tanpa network (0.00s).

✓ Signature NewProcessor(PaymentGateway) eksplisit dan terverifikasi.

✓ Missing dependency gagal di compile-time jika argumen kosong.

Shift dependency discovery from hidden runtime lookups to strict compile-time contracts.

Bedah Anatomi: Service Locator Anti-Pattern

Mengapa menyuntikkan Container ke kelas domain dianggap buruk oleh standar PSR-11 & Fowler.

```
type BadProcessor struct {
    container Container // Anti-Pattern: Mengikat ke container API
}

func (p *BadProcessor) ProcessPayment(amount int) error {
    // Opaque lookup: pembaca kode tidak tahu dependensi asli
    return p.container.GetPaymentGateway().Charge(m)
}
```

Service Locator tampak fleksibel, tetapi ia mengikat kelas bisnis ke API kontainer dan menyembunyikan dependensi sebenarnya dari siapapun yang membaca signature kelas.

Container injection pollutes domain logic with infrastructure orchestration concerns.

internal/di/locator.go:10-25

Value Objects Tidak Boleh Menggunakan DI

Membedakan service berinfrastruktur dari struktur data murni tanpa side-effect.

: ()
: ()
: ()

DI dicadangkan untuk service yang berinteraksi dengan infrastruktur eksternal.
Struktur data murni seperti Money cukup dibuat langsung tanpa melibatkan container.

Inject behavior; instantiate state.

Strategi Mocking: Pengujian Cepat & Terisolasi

Test double menguji interaksi kontrak tanpa memerlukan infrastruktur pihak ketiga.

```
type MockGateway struct {
    ChargedMoney di.Money
    ShouldFail   bool
}

func (m *MockGateway) Charge(money di.Money) error {
    if m.ShouldFail { return errors.New("gateway unavailable") }
    m.ChargedMoney = money
    return nil
}
```

Dengan menginjeksi mock sederhana di unit test, kita dapat memverifikasi payload transaksi dan simulasi error tanpa menyentuh jaringan sama sekali.

Lightweight in-memory test doubles verify contract adherence in microseconds.

tests/processor_test.go:10-22

Batas Integritas: Apa yang Tidak Diuji

Keberhasilan test suite unit tidak menjamin kesiapan performa di skala produksi.

DI LUAR CAKUPAN PENGUJIAN LAB

Ketahanan koneksi TCP/HTTP saat RealGateway menghadapi latency spike.
Penyelesaian cyclic dependency kompleks antar puluhan service.
Manajemen memori lifecycle singleton vs transient under high load.
Kecepatan eksekusi compile-time vs reflection-based DI containers.

Suite test memverifikasi isolasi logika unit dan propagasi error. Pengujian ini tidak membuktikan ketahanan jaringan produksi atau manajemen memori container skala besar.

Unit test passing confirms logical contracts; production resiliency requires integration stress tests.

Risiko Produksi: Constructor Bloat & Over-Injection

Banyaknya dependensi di konstruktor adalah 'code smell' pelanggaran Single Responsibility.

: ()
: ()
: ()

Jika konstruktor Anda membutuhkan belasan parameter, jangan cari framework auto-wiring untuk menyembunyikannya. Pecah kelas tersebut karena ia melanggar Single Responsibility Principle.

Constructor parameter explosion is a distress signal that the class has too many responsibilities.

Checklist Adopsi Dependency Injection

Panduan langkah demi langkah menerapkan DI yang bersih pada repositori tim.

- ✓ Deklarasikan dependensi sebagai interface minimal di dekat consumer.
- ✓ Gunakan Constructor Injection sebagai mekanisme default (valid-at-birth).
- ✓ Pusatkan instansiasi konkret di Composition Root (main.go).
- ✓ Larang passing Container atau Service Locator ke domain logic.
- ✓ Instansiasi Value Object (Money, DTO) secara langsung tanpa DI.
- ✓ Pastikan semua unit test berjalan terisolasi menggunakan Test Double.

Enforce explicit constructors, isolate composition roots, and keep domain logic container-free.

Anti-Pattern: Jebakan Umum dalam Praktik DI

Pola keliru yang sering muncul saat tim mencoba menerapkan Inversion of Control.

JEBAKAN ARSITEKTUR UMUM

Framework Lock-in: Memasukkan decorator / tag framework ke domain core.
Service Locator Terselubung: Mengoper map[string]any sebagai dependency bag.
Interface Bloat: Membuat interface 1-to-1 untuk struct yang tidak pernah di-swap.
Ambient Context: Menggunakan global singleton yang dimutasi saat testing.

CARA REPRODUKSI MASALAH

Inject mock yang mengembalikan error -> verifikasi penanganan error.
Kirim amount <= 0 -> verifikasi gateway mock tidak menerima hit sama sekali.

Hindari mengotori logika domain dengan API framework. Gunakan interface Go yang idiomatis di sisi konsumen dan jangan membuat abstraksi prematur untuk objek yang tidak perlu di-mock.

Do not invent abstractions without test or substitution pressure.

Reproduksi Mandiri & Command Runner

Verifikasi seluruh konsep di mesin lokal menggunakan Go standard library.

```
cd labs/16-dependency-injection
go test -v ./...       # Menjalankan 6 test cases (mock isolation)
go test -race ./...    # Verifikasi data race pada goroutine
go run ./cmd/demo      # Menjalankan demo perbandingan kedua pola
```

Eksperimen ini zero-external-dependency dan dapat direproduksi dalam waktu kurang dari satu detik pada mesin lokal manapun.

Zero-dependency architecture enables sub-second reproducible verification.

Prinsip Inti: Program to Interfaces, Not Implementations

Arsitektur yang tangguh dibangun di atas batas kontrak yang jelas dan modular.

: ()
: ()
: ()

Prinsip sejati DI bukan tentang framework atau anotasi canggih, melainkan tentang memisahkan concern: biarkan composition root merakit, dan biarkan modul domain mengeksekusi.

True inversion of control liberates domain services from infrastructure assembly.

Pisahkan konfigurasi dari penggunaan; suntikkan kontrak, bukan container.

Program to interfaces, wire at the composition root, and let domain logic remain pure.

Software Engineering Lab 16 – Dependency Injection & Inversion of Control

internal/di/gateway.go • internal/di/processor.go • internal/di/locator.go
cmd/demo/main.go • tests/processor_test.go
README.md • content/02-master-draft.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/16-dependency-injection`

Lukman / lukman-ss

Commit: 9dd0553 • Status: APPROVED
Test Suite: 6/6 PASS (0.00s execution per case)
Race Safety: Clean (go test -race ./...)
Batasan: Manual wiring demo; bukan evaluasi container lifecycle production.